

Autómatas, Teoría de Lenguajes y Compiladores

Proyecto especial - DELTA GIRLS FORCE

Integrantes:

- Percich, Luana - 64316 - lpercich@itba.edu.ar
 - Petrikovich, María Belén - 63689 - mpetrikovich@itba.edu.ar
 - Trajterman, Catalina - 64175 - ctrajterman@itba.edu.ar
-

Introducción

El objetivo de este trabajo práctico fue desarrollar un compilador capaz de transformar consultas del Álgebra Relacional representadas en JSON en sentencias SQL válidas. La idea principal fue ofrecer una forma sencilla para que el usuario pueda expresar operaciones del Álgebra Relacional y obtener como resultado una consulta SQL equivalente, aprovechando la potencia y practicidad de este lenguaje.

Si bien el compilador está implementado en C, se utilizaron dos herramientas fundamentales: Flex y Bison. Flex se empleó para construir el analizador léxico, encargado de leer el archivo JSON de entrada y dividirlo en tokens reconocibles para el compilador. Por su parte, Bison se utilizó para generar el analizador sintáctico, responsable de verificar la estructura de esos tokens según la gramática definida y construir el Árbol de Sintaxis Abstracta (AST) que representa la consulta del usuario. Finalmente, a partir de ese AST el generador recorre los nodos del Álgebra Relacional y produce el SQL final, permitiendo así pasar de una representación conceptual a una consulta concreta.

El trabajo fue dividido en 3 etapas. La primera etapa fue la etapa de diseño, donde se definieron los aspectos importantes como los requerimientos y la propuesta. Luego de haber obtenido un feedback, se comenzó a trabajar con la segunda etapa: la implementación del frontend (analizadores léxico y sintáctico) para la construcción del AST mencionado anteriormente.

Finalmente, se concluyó el trabajo con la tercera etapa: el backend. En ella se implementaron las validaciones y generación de código necesarios para el correcto funcionamiento.

Modelo computacional

Dominio

El dominio del compilador desarrollado se centró en la traducción de expresiones del Álgebra Relacional a consultas SQL válidas. Para ello, se definió un lenguaje de entrada basado en estructuras JSON que representan las operaciones del Álgebra Relacional: selección, proyección, renombrado, producto cartesiano, unión, intersección, diferencia, agregaciones, agrupamientos y join. Se decidió que el enfoque sea más bien acotado, centrándose en consultas de lectura, y sin contemplar operaciones de modificación como `INSERT`, `UPDATE` o `DELETE`.

Dentro del dominio se prestó especial atención al manejo de consultas compuestas y estructuras anidadas, propias del Álgebra Relacional. Cada operación (σ , π , ρ , $\bowtie$, $\cup$, $\cap$, $-$, $\times$, γ) puede aparecer de forma recursiva, y el compilador debe ser capaz de interpretarlas y combinarlas respetando el orden semántico de las mismas. Esto se ve reflejado en la posibilidad de que una selección se aplique sobre una proyección, o que una agregación incluya un group-by y múltiples funciones.

La representación en JSON facilitó enormemente la lectura de las expresiones, ya que es un lenguaje sencillo de interpretar, y además de esta manera se evitó la dificultad de interpretar los símbolos propios del Álgebra.

Por último, a partir del AST el generador de código SQL recorre el dominio definido por el usuario y produce una consulta equivalente que puede ser ejecutada en cualquier motor SQL estándar.

Lenguaje

Como se mencionó en la sección anterior, las consultas son ingresadas en formato JSON para facilitar la lectura. Se implementaron claves para generar una relación 1-1 con el SQL para poder generar la consulta correspondiente con más facilidad. Además, ingresar las consultas en JSON permitió manejar con

facilidad los casos de consultas anidadas. Ambos aspectos mencionados pueden visualizarse en el siguiente ejemplo:

```
{
  "UNION":{
    "left": {
      "PROJECTION":{
        "attributes": ["nombre"],
        "input": {
          "SELECT":{
            ">":["edad", 30],
            "input": {
              "TABLE": "Persona"
            }
          }
        }
      }
    },
    "right": {
      "PROJECTION":{
        "attributes": ["nombre"],
        "input": {
          "SELECT":{
            "<":["edad", 20],
            "input": {
              "TABLE": "Persona"
            }
          }
        }
      }
    }
  }
}
```

Por último, se consideró que dentro del lenguaje de entrada existen claves obligatorias y claves opcionales, dependiendo del tipo de operación

representada. Algunas construcciones requieren necesariamente ciertos campos para que la consulta tenga sentido, mientras que otras no precisan estas complejidades. Por ejemplo, en una Proyección la lista de atributos es obligatoria, porque sin ella no es posible determinar qué columnas se deben conservar. En cambio, en un Producto Cartesiano, no se requiere ninguna condición adicional: basta con indicar las dos expresiones que participan, por lo que no necesita claves extras.

Implementación

Para implementar el compilador se utilizó el proyecto provisto por la cátedra como base. Luego, se adaptó el dominio para que coincida con el de Álgebra Relacional del JSON. Este desarrollo fue dividido en dos: el Stage II en la que se focalizó en la implementación del frontend y el Stage III, donde se hizo lo propio con la parte del backend.

Frontend

El frontend también se encuentra dividido en dos partes: el análisis léxico y el sintáctico. Estos se encargan de reconocer la estructura del lenguaje JSON y construir el árbol sintáctico de consultas del Álgebra Relacional.

Analizador léxico

Los patrones léxicos de los elementos del lenguaje JSON fueron definidos en `FlexPatterns.l`. Dentro de este archivo se definieron las palabras claves de las operaciones de Álgebra Relacional: `SELECT`, `PROJECT`, `JOIN`, etc. Además, se definieron los símbolos estructurales como `{`, `}`, `[`, `]`, `(`, `)`. Por último, se manejaron los distintos tipos de tokens.

Las acciones correspondientes a los patrones fueron implementadas en `FlexActions.c`, donde se realiza la definición de los tokens y el envío de los mismos al analizador sintáctico, además del manejo de contexto para las llaves, corchetes y paréntesis.

Analizador sintáctico

La estructura gramatical del lenguaje fue definida en `BisonGrammar.y`, para describir el formato JSON representando las consultas del Álgebra Relacional. Se definieron nodos terminales y no terminales, tales como `program`, `expression` y

`condition` . Además, se hizo uso de listas auxiliares para tratar, por ejemplo, los atributos y funciones de agregación.

La idea principal es que las operaciones del Álgebra Relacional puedan ser anidadas unas dentro de otras, tal como funciona el lenguaje en sí mismo. Por lo tanto, la gramática está diseñada para reflejar esta estructura recursiva, al igual que el AST.

Todas las expresiones, excepto `BASE_TABLE` , que en el Álgebra Relacional representa directamente el nombre de la relación sobre la que se está haciendo la operación, contienen internamente otra expresión. Puede decirse que la gramática forma una especie de “*mamushka*” de expresiones, lo que permite modelar consultas anidadas de manera natural y estructurada.

Backend

El objetivo de la implementación del backend fue validar que las consultas hechas en JSON sean correctas, y posteriormente generar una salida consistente con la consulta SQL. Esto asegura que la estructura no solo sea correcta desde el punto de vista formal, sino también desde el sentido lógico. Al igual que el frontend, el backend está dividido en dos partes principales.

En primer lugar, dentro del archivo `SQL.c` se hace la verificación semántica de las expresiones enviadas en la entrada. Aquí se revisa que las operaciones del Álgebra Relacional respeten sus reglas: que las proyecciones contengan atributos válidos, que las operaciones binarias reciban entradas compatibles, que las agregaciones y sus `group_by` sean consistentes, entre otras verificaciones.

Por otro lado, el archivo `Generator.c` es el encargado de transformar el AST en una sentencia SQL final. La generación se realizó de forma recursiva, respetando la estructura interna de cada operación, lo que permitió manejar consultas anidadas de manera natural. Si bien los campos de la entrada ya fueron validados en `SQL.c` , dentro de `Generator.c` se vuelve a verificar que los parámetros de las funciones no sean nulos. Esta decisión se tomó porque, más allá de que “*no deberían ser nulos*”, resulta una buena práctica defensiva que mejora la robustez del sistema y facilita el trabajo ante cambios futuros.

Limitaciones

Una limitación de esta implementación es que en SQL se permiten operaciones binarias sin que los nombres de las columnas coincidan necesariamente, pero

sí con la condición de que sus tipos sean compatibles. Como el lenguaje de entrada no incluye los tipos de los atributos, no es posible validar que dos columnas con nombres distintos posean el mismo tipo. Por lo tanto, se optó por restringir que las operaciones binarias que contengan proyecciones anidadas proyecten necesariamente los mismos atributos en ambos lados de la operación.

Otra limitación es que el compilador está pensado para SQL Postgres versión mayor a la 16, ya que en la operación `DIFFERENCE` se utiliza `EXCEPT`, y en versiones anteriores es obligatorio el uso de alias para subqueries, algo que esta implementación no contempla.

Una tercera limitación es que si una operación de agregación está anidada con una de selección, las columnas involucradas en la condición de selección deben pertenecer al `group_by`. Caso contrario, no serán parte del `SELECT` interno, lo que genera una consulta SQL incorrecta. Para futuras extensiones se podría permitir incluir automáticamente estas columnas, o validar funciones de agregación adicionales.

Por último, para comparar columnas con strings, los constantes deben estar entre comillas simples y dobles (es decir, `"string"`), tanto por la sintaxis del JSON como por la necesidad de distinguir entre identificadores y literales.

Dificultades encontradas

A lo largo de la implementación del trabajo práctico, distintas dificultades fueron encontradas. En cuanto a la primera etapa, las dificultades se centraron en cómo expresar el Álgebra Relacional de manera sencilla para que posteriormente el compilador lo interprete. Finalmente, la idea de utilizar JSON fue la mejor, y de hecho evitó muchos problemas a futuro.

En la segunda etapa fue donde más dificultades se encontraron. Al ser Flex y Bison algo nuevo que aprender, el desarrollo de esta parte resultó tediosa, principalmente los primeros días. De todas maneras, estas dificultades pudieron solucionarse a tiempo, en parte gracias a los ayudantes de la materia.

Si bien en la tercera etapa el trabajo ya estaba más consolidado, se tuvo que aprender a administrar correctamente el tiempo. Además, en esta última etapa también se tuvieron que corregir algunos aspectos de la segunda etapa, por lo que se sumaron tareas nuevas para hacer.

Correcciones del Stage II

Se intentaron corregir la mayor cantidad de observaciones de la implementación del Stage II. Sin embargo, se optó por mantener la implementación original en dos casos.

En primer lugar, se decidió continuar entrando al contexto luego de hacer un push. Esto se debe a que se decidió priorizar otras cosas, dado que esa implementación funciona correctamente y solo se utiliza para verificar que todas las llaves y paréntesis queden cerrados.

Por otro lado, tanto en las funciones de agregación como en las de comparación, se decidió mantener los strings en lugar de convertirlos en enums, ya que luego esos strings pasan a la salida directamente, sin pasar por un `strcmp`. De todas maneras, antes de tomar esta decisión la opinión del profesor fue consultada y considerada.

Futuras extensiones

Si bien el compilador cumple con el objetivo principal de traducir expresiones del Álgebra Relacional a SQL, existen varias ideas que permitirían ampliar sus capacidades y volverlo más completo. Una posible extensión es incorporar soporte para operaciones de modificación de datos, como `INSERT`, `UPDATE` y `DELETE`. Hasta el momento, el proyecto se centró exclusivamente en consultas de lectura, pero sumar estas funcionalidades permitiría abarcar una mayor parte del lenguaje SQL y acercarse a un sistema más completo.

Otra dirección natural de mejora es ampliar el conjunto de operaciones del Álgebra Relacional soportadas, incluyendo nuevas variantes de joins, tipos de datos adicionales o funciones de agregación extendidas.

Finalmente, podría explorarse la posibilidad de generar SQL adaptado a distintos motores de bases de datos (como MySQL o SQLite), permitiendo adaptar detalles sintácticos o comportamientos particulares según la selección del usuario.

Conclusiones

El desarrollo de este compilador permitió integrar conceptos de diseño de lenguajes, análisis léxico y sintáctico, construcción de ASTs y generación de código. A lo largo del proyecto se comprobó cuán acertada fue la decisión de

representar el Álgebra Relacional en formato JSON, ya que el mismo facilitó la lectura y el manejo de consultas anidadas, entre otros aspectos.

Asimismo, si bien la adaptación a Flex y Bison significó un desafío inicialmente, permitió adquirir un dominio práctico sobre estas herramientas y entender cómo se estructura un compilador de verdad. Posteriormente, el desarrollo del backend permitió consolidar este aprendizaje transformando el AST en consultas SQL válidas.

A pesar de las limitaciones mencionadas anteriormente, se considera que el compilador cumple satisfactoriamente su objetivo principal: transformar consultas expresadas en Álgebra Relacional en JSON a SQL estándar. Estas restricciones marcan un claro camino para futuras extensiones del programa.

En conjunto, este trabajo práctico permitió aplicar conceptos teóricos en un entorno práctico, fortaleciendo la comprensión del proceso de compilación y permitiendo continuar explorando técnicas de diseño de lenguajes.

Referencias y bibliografía utilizada

- Aho, A., Lam, M., Sethi, R. & Ullman, J. — *Compilers: Principles, Techniques, and Tools*. 2nd Ed. Pearson.
- Documentación oficial de Flex — <https://westes.github.io/flex/manual/>
- Documentación oficial de Bison — <https://www.gnu.org/software/bison/manual/>
- Especificación JSON — <https://www.json.org/json-es.html>
- PostgreSQL 16 Documentation — <https://www.postgresql.org/docs/16/>